

Programming 'Java' { [For first Capstone]

// by Maysun

}

01 {

[Rating System]

< Post Rating >

}

Logic

```
1
2
3 First, I validate that the rating is between 0 and 5
4 • Then I check if:
5 • the user exists
6 • the merchant exists
7 • the product exists If all conditions are valid:
8 • I set the rating to the product
9 • Otherwise:
10 • I return a specific error based on the issue
11
12
13
14
```

Rating System; {

```
//extra endpoint 7 . rating
public int rating(String userId, String merchantId, String productId, int rating){ 1 usage
    if(rating<0 || rating>5){
        return -1;
    }
    User user = userService.userExist(userId);
    if (user == null) {
        return -2;
    }
    Merchant merchant = merchantService.merchantExist(merchantId);
    if (merchant == null) {
        return -3;
    }
    Product product = productService.productExist(productId);
    if (product == null) {
        return -4;
    }
    product.setRating(rating); //Rating

    return 1;
}
```

02 {

[Static System]

< Users + Products + Stock >

}

Logic

1

2

3

4

5

6

- Count total users

7

- Count total products

8

- Calculate total stock using a for loop

9

10

11

12

13

14

Static System; {

```
// 8 static system
    public int getUsersCount(UserService userService) { 1 usage
        return userService.getUsers().size();
    }

    public int getProductsCount() { 1 usage
        return products.size();
    }

    public int getTotalStock(MerchantStockService merchantStockService) { 1 u
        int total = 0;
        for (MerchantStock m : merchantStockService.getMerchantStocks()) {
            total += m.getStock();
        }
        return total;
    }
}
```

```
1 03 {
```

```
2  
3  
4  
5 [Filter By Price]
```

```
6  
7  
8 < get Filer by min and max>
```

```
9  
10  
11  
12 }  
13  
14
```

Logic

- 1
- 2
- 3
- 4 • Take minimum and maximum price as input
- 5 • Loop through all products
- 6 If:
- 7 • $\text{price} \geq \text{min}$
- 8 • $\text{price} \leq \text{max}$
- 9 • → add the product to the Products List
- 10
- 11
- 12
- 13
- 14

Filter By Price; {

```
//extra point 4. filter by price
public List<Product> byPrice(int min, int max) { 1 usage
    List<Product> products1 = new ArrayList<>();
    for(Product p : products){
        if (p.getPrice() >= min && p.getPrice() <= max) {
            products1.add(p);
        }
    }
    return products1;
}
```

04 {

[Available Product]

< get Available product in
stock >

}

Logic

- Loop through all merchant stocks
- If stock > 0 → product is available
- the merchant exists
- Get the product using productService
- Add it to the list
- Stop when reaching the limit

Available Product;

```
1  {
2
3      //11. available product in stock
4      public ArrayList<Product> availableProducts(int limit) { 1 usage
5          ArrayList<Product> av = new ArrayList<>();
6          for (MerchantStock m : merchantStocks) {
7              if (m.getStock() > 0) {
8                  Product product = productService.productExist(m.getProductId());
9                  if (product != null && !av.contains(product)) {
10                      av.add(product);
11                  }
12                  if (av.size() == limit) {
13                      break;
14                  }
15              }
16          }
17          return av;
18      }
19  }
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14

05 {

[Max Price]

< get Max Price Product >

}

Logic

- If the list is empty → return null
- Start with the first product as the initial max
- Loop through all products
- If a product has a higher price:
 - update max
- Return the most expensive product

Max Price Product;

```
1  {
2      //10 Expensive product
3      public Product getMaxPriceProduct() { 1 usage
4          if (products.isEmpty()) {
5              return null;
6          }
7          Product max = products.get(0);
8          for (Product p : products) {
9              if (p.getPrice() > max.getPrice()) {
10                 max = p;
11             }
12         }
13         return max;
14     }
```

```
1 PostMapping("/the  
2 End"){  
3  
4
```

```
5  
6     public String thank(String  
7       Thank, String You){  
8       // Thank you all  
9  
10      }  
11
```

```
12 }  
13  
14
```

